

PARTE - A

6.26 - Resposta: A diferença fundamental entre uma fila e uma lista cadastral é que na lista cadastral, os elementos são removidos pelo seu valor e não por sua posição, ou seja, enquanto na fila o primeiro valor recebido é o primeiro a sair, na lista cadastral, o valor recebido será um valor específico que deverá ser informado, e não sua posição.

6.19 - Resposta: Código em C++ da parte B

PARTE - B

ListaEncadeada.h

```
#include <iostream>
```

```
using namespace std;
```

```
struct Node {  
    int valor;  
    Node* next;  
};
```

```
class ListaCadastral {  
private:  
    Node* start;  
    Node* current;  
  
    bool exists(int element) {  
        Node* aux = this->start;  
        while (aux != NULL) {  
            if (aux->valor == element) {  
                return true;  
            }  
            aux = aux->next;  
        }  
        return false;  
    }  
}
```

```
public:  
    ListaCadastral() {  
        this->start = NULL;  
        this->current = NULL;  
    }
```

```
}
```

```
bool vazia() {  
    return this->start == NULL;  
}
```

```
bool getFirst(int& first_element) {  
    if (!this->vazia()) {  
        first_element = this->start->valor;  
        this->current = this->start;  
        return true;  
    } else {  
        return false;  
    }  
}
```

```
bool getNext(int& element) {  
    if (this->current != NULL) {  
        this->current = this->current->next;  
        if (this->current != NULL) {  
            element = this->current->valor;  
            return true;  
        }  
    }  
    return false;  
}
```

```
bool insere(int element) {  
    if (this->exists(element)) {  
        return false;  
    }  
    if (this->start == NULL) {  
        this->start = new Node;  
        this->start->valor = element;  
        this->start->next = NULL;  
        return true;  
    }  
    Node* aux = new Node;  
    if (element < this->start->valor) {  
        aux->valor = element;  
        aux->next = this->start;  
        this->start = aux;  
        return true;  
    }  
}
```

```

    aux = this->start;
    Node* prev = NULL;
    while (aux->valor < element && aux->next != NULL) {
        prev = aux;
        aux = aux->next;
    }

    Node* newNode = new Node;
    newNode->valor = element;
    if (element > aux->valor) {
        if (aux->next == NULL) {
            newNode->next = NULL;
        } else {
            newNode->next = aux->next;
        }
        aux->next = newNode;
    } else {
        newNode->next = aux;
        prev->next = newNode;
    }

    return true;
}

bool remove(int element) {
    if (exists(element)) {
        Node* aux = this->start;
        Node* previous = NULL;
        while (aux->valor != element) {
            aux = aux->next;
            if (previous == NULL) {
                previous = this->start;
            } else {
                previous = previous->next;
            }
        }

        if (this->current == aux) {
            this->current = this->start;
        }

        previous->next = aux->next;
        delete aux;
    }
}

```

```

        return true;
    } else {
        return false;
    }
}
};

```

main.cpp

```

#include <iostream>
#include "listacadastral.h"

using namespace std;

void mostra(ListaCadastral& lista) {
    cout << endl;
    int current;
    bool res = lista.getFirst(current);
    if (!res) {
        cout << "Lista vazia" << endl;
        return;
    }

    while (res) {
        cout << current << endl;
        res = lista.getNext(current);
    }
    cout << "Fim da lista" << endl;
    cout << endl;
}

int main() {
    ListaCadastral l = ListaCadastral();

    l.insere(2);
    mostra(l);
    l.insere(5);
    mostra(l);
    l.insere(10);
    cout << "Elementos inseridos" << endl;
    mostra(l);

    bool res;
}

```

```
res = l.remove(3);  
cout << "res=" << res << endl;  
mostra(l);
```

```
res = l.remove(5);  
cout << "res=" << res << endl;  
mostra(l);
```

```
l.insere(1);  
mostra(l);  
l.insere(3);  
mostra(l);  
l.insere(6);  
mostra(l);  
l.insere(4);  
mostra(l);  
l.remove(2);  
mostra(l);
```

```
}
```